

Automaton Auditor · Multi-Agent AI System

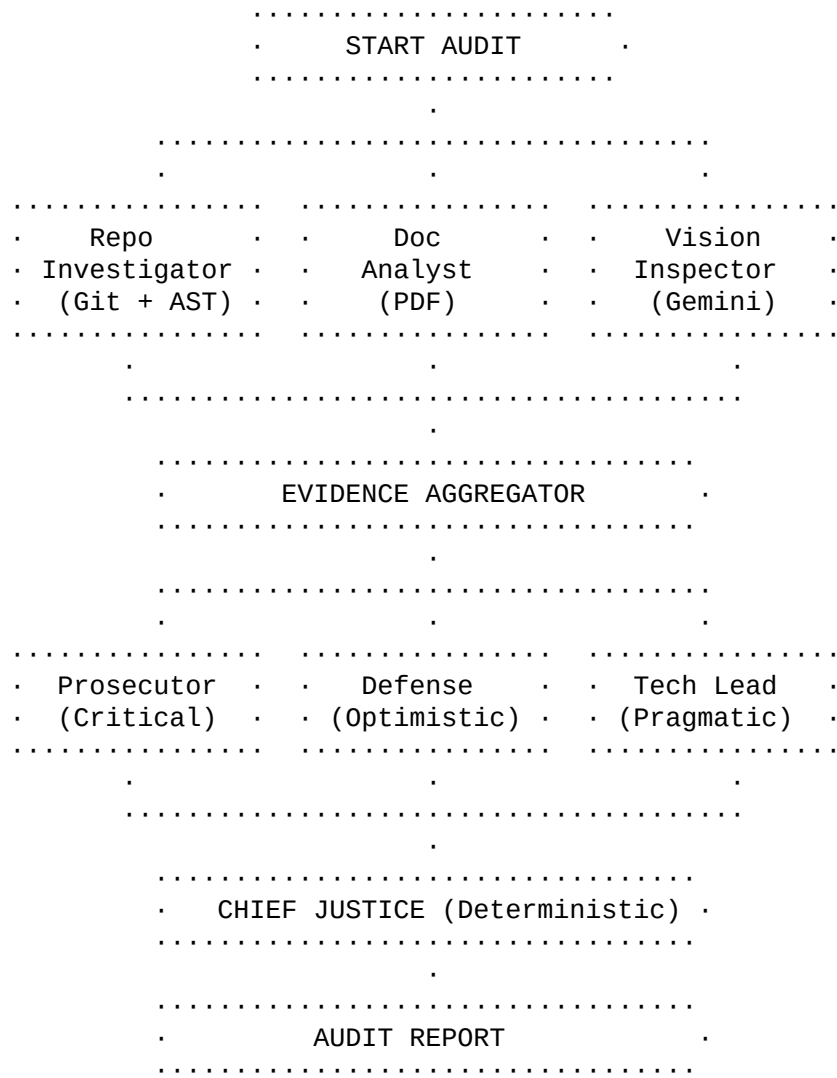
Author: Amir Ahmedin | **GitHub:** github.com/IbnuEyni

What It Does

A 7-agent "Digital Courtroom" that autonomously audits code repositories. Three detective agents collect evidence in parallel, three judicial agents evaluate dialectically, and one chief justice synthesizes a final verdict deterministically.

Performance: Audits a full repository in ~45 seconds for \$0.02.

Architecture



Multi-Agent Coordination

| Layer | Agents | Role | Pattern | |-----|-----|-----|-----| | Detective | 3 agents | Forensic evidence collection |
Fan-out (parallel) | | Judicial | 3 judges | Dialectical evaluation | Fan-out (parallel) | | Supreme Court | 1 justice |
Deterministic synthesis | Fan-in (merge) |

Key Design Decisions

- **LangGraph StateGraph** · declarative workflow with compile-time validation
 - **Fan-out / Fan-in parallelism** · 2.5x speedup over sequential execution
 - **State reducers (CRDTs)** · conflict-free parallel writes without locks
 - **Deterministic synthesis** · hardcoded Python rules, NOT an LLM prompt
 - **Sandboxed execution** · all git operations in isolated temp directories
-

Synthesis Rules

- 1 **Security Override** · security flaws cap the score at 3
 - 2 **Fact Supremacy** · Tech Lead overrides when score · 2
 - 3 **Weighted Resolution** · Tech Lead 50%, Prosecutor 30%, Defense 20%
-

Tech Stack

Python | LangGraph | Pydantic | Groq (Llama 3.3 70B) | Google Gemini (Vision) | LangSmith (Observability)

Results

- 7 specialized agents coordinated through a single state graph
- 12 judicial opinions per audit (3 judges × 4 criteria)
- Full observability via LangSmith tracing
- Type-safe throughout (100% Pydantic validation)